GATE DA 2027 / GATE LAB / PYTHON

The revision companion

From first principles to confident Python reasoning.

41 lessons / 12 modules / 120 topic questions / 60 flashcards

Use this alongside the interactive guide. Read a prompt, close the notes, explain the rule, and trace a fresh example. The guide contains full explanations, editable examples, exercises and verified PYQ references.

Scope and study order

Core Python: modules 1-7. Practical Python: modules 8-9. DSA bridges: modules 10-11. Applied projects: module 12. This is the Python revision companion. The full DSA track, including all five named sorting algorithms, tree operations and shortest paths, has its own companion in the same site.

Scope verified on 24 October 2026 against the official GATE DA 2027 Section 4 syllabus PDF. All named programming and DSA topics are mapped across the two tracks. Other DA subjects remain separate.

Updated 24 October 2026. Python 3 semantics; examples avoid dependence on CPython object interning. Questions and explanations are original unless explicitly labelled as an official PYQ.

Your revision route

01. Think like Python

Core Python / 4 lessons

02. Control the flow

Core Python / 3 lessons

03. Work with text

Core Python / 4 lessons

04. Lists, tuples & memory

Core Python / 3 lessons

05. Dictionaries & sets

Core Python / 3 lessons

06. Functions, scope & recursion

Core Python / 5 lessons

07. Write expressive Python

Core Python / 3 lessons

08. Build reliable programs

Practical extension / 4 lessons

09. Objects & advanced tools

Practical extension / 3 lessons

10. Reason about algorithms

DSA bridge / 3 lessons

11. Data structures in Python

DSA bridge / 3 lessons

12. Build, explain, revise

Hands-on studio / 3 lessons

Think like Python

Execution, values, expressions and the first reliable mental model

01 / From a file to a running program

- A program is a sequence of instructions that transforms input into output.

- Save code as a .py file and run it with python filename.py (or py filename.py on Windows).

- Indentation defines blocks.

- Syntax errors prevent parsing.

Trap to remember: 10% is written as 0.10, not 10. Python does not infer units such as rupees or percentages.

Try it: Change the quantity to 5 and discount to 20%. Predict the total before running.

02 / Names, objects and types

- A name refers to an object; it is not a permanently typed storage box. x = 3 binds x to an integer object. x = "three" later rebinds that name to a string.

- The main scalar types are int (arbitrary precision within memory limits), float (binary floating point), bool (True or False), complex and NoneType.

- Use type(x) to inspect a type and isinstance(x, T) for a type relationship. bool is a subclass of int, so isinstance(True, int) is True.

- Parallel assignment evaluates the right side before binding the left side.

Trap to remember: Do not use is to compare numeric or string values. Object reuse is an implementation detail; use == for equal values.

Try it: Start with x=2,y=5. Swap them without a temporary name, then print their types.

03 / Arithmetic and the negative-number trap

- Use +, -, *, /, //, %, and ** for addition, subtraction, multiplication, true division, floor division, remainder, and exponentiation.

- For nonzero b, integer division obeys a == (a // b) * b + a % b.

- Exponentiation associates right-to-left: 2 ** 3 ** 2 is 2 ** 9.

- Binary floats approximate most decimal fractions.

Trap to remember: / and // are not interchangeable. Also, ^ is bitwise XOR, not exponentiation.

Try it: For a=7,b=-3, calculate quotient and remainder by hand, then check the identity.

04 / Comparisons, truth and bits

- Comparisons ==, !=, <, <=, > and >= produce Booleans.

- False values include False, None, numeric zero and empty containers.

- A practical precedence ladder is: indexing/calls; exponentiation; unary arithmetic; multiplication/division/remainder; addition/subtraction; shifts; bitwise &, ^, |; comparisons and membership/identity; not; and; or.

- For integers, & intersects set bits, | combines them, ^ toggles differing bits, and << or >> shifts.

Trap to remember: A fallback such as value or default replaces all false values, including legitimate zero. Use an explicit None check when zero must survive.

Try it: Write a safe test for a nonempty list whose first element is positive.

Control the flow

Decisions, iteration, tracing and correctness

01 / Decisions and input contracts

- An if/elif/else chain selects the first true branch and skips the remaining branches.

- input() returns a string, even when the user types digits.

- Express preconditions explicitly.

- A conditional expression is value_if_true if condition else value_if_false.

Trap to remember: if x == 1 or 2 is almost always wrong: it means (x == 1) or 2 and is always truthy. Use x in (1, 2).

Try it: Classify a year as leap if divisible by 400, or divisible by 4 but not 100. Test 1900, 2000, 2024.

02 / for, range and while

- A for loop takes successive values from an iterable. range(start, stop, step) is a compact sequence of integers excluding stop.

- A while loop repeats while its condition is true.

- An accumulator combines values (sum, product, maximum); a counter counts events.

- An invariant is a statement that stays true before each iteration.

Trap to remember: range endpoints are excluded. range(1, n) does not include n. An empty range executes the body zero times.

Try it: Compute the sum of odd integers from 1 to 99 with a loop. Then derive it without a loop.

03 / break, continue, else and nested loops

- break exits the nearest enclosing loop. continue skips the rest of the current iteration and proceeds to the next one. pass does nothing; it is a syntactic placeholder, not a skip instruction.

- A loop else runs when the loop finishes without break, including when the loop executes zero times.

- For nested loops, draw one row per outer-loop value and count inner iterations.

- Avoid changing list length while iterating over it: shifted positions can be skipped.

Trap to remember: The else belongs to the loop at the same indentation, not the inner if.

Try it: Return the first negative number in a list, or None if no negative exists.

Work with text

Indexing, slicing, parsing and clean transformations

01 / Strings and slicing

- A str is an immutable sequence of Unicode code points.

- s[start:stop:step] selects indices beginning at start and ending before stop.

- s[::-1] reverses the sequence. s[-1:0:-1] includes the last character and excludes index 0.

- Strings cannot be edited in place.

Trap to remember: A slice never changes the source string. Negative step does not automatically reverse arbitrary start/stop choices.

Try it: Extract every second character from "abcdefgh" starting at index 1, then reverse the result.

02 / Methods, parsing and formatting

- strip removes specified characters from the two ends, not a substring from the middle. split() with no separator collapses runs of whitespace; split(",") treats each comma as a delimiter and can preserve empty fields. separator.join(strings) assembles strings with separators.

- lower and casefold support case normalisation; casefold is more aggressive for Unicode comparisons. startswith, endswith, find, count and replace support common text tasks. find returns -1 when absent; index raises ValueError.

- f-strings evaluate expressions inside braces.

- Parsing is a pipeline: normalise, split, validate, convert, then compute.

Trap to remember: strip("ab") removes any a or b at either end; it does not remove exactly the prefix "ab". Use removeprefix for that purpose.

Try it: Normalise " Learn PYTHON today " into "learn python today".

03 / Text algorithms and edge cases

- A palindrome reads identically forwards and backwards.

- A character histogram maps each character to a count.

- A two-pointer palindrome check advances from both ends and can stop at the first mismatch.

- For repeated concatenation of many strings, collect pieces in a list and join once.

Trap to remember: The empty string is usually a palindrome under the standard definition; document that decision rather than allowing a test to surprise you.

Try it: Count vowels in text without counting uppercase vowels differently.

04 / Unicode, bytes and safe conversion

- Text and binary data are different abstractions. str represents Unicode code points; bytes represents integers from 0 to 255.

- A character can occupy multiple bytes in UTF-8.

- ord converts a one-character string to its code point; chr performs the inverse.

- Specify encoding explicitly for portable file processing.

Trap to remember: Do not assume a character is one byte, or that len counts visual grapheme clusters.

Try it: Round-trip the string "naïve" through UTF-8 and compare its text length with its byte length.

Lists, tuples & memory

Mutation, copying and the traps behind output questions

01 / Sequence operations and list methods

- A list is a mutable ordered sequence; a tuple is an immutable ordered sequence.

- append(x) adds one object; extend(iterable) appends its elements. insert(i,x) inserts before index i. pop(i) removes and returns an item; remove(x) removes the first equal value. del can delete an index or slice. clear empties the list.

- sort and reverse mutate a list and return None. sorted(iterable) creates a new list.

- Sequence unpacking requires matching lengths unless a starred target absorbs remaining elements: first, *middle, last = values.

Trap to remember: a = a.sort() loses the reference to the sorted list by rebinding a to None.

Try it: Remove duplicates from [3,1,3,2,1] while preserving the first occurrence order.

02 / Aliasing, shallow and deep copies

- a = [1,2]; b = a gives two names for one list. b.append(3) changes the shared object. b = [9] instead rebinds b to a different object and leaves a unchanged.

- b = a[:] or a.copy() creates a new outer list, but nested objects remain shared.

- Equality == asks whether values compare equal; identity is asks whether the objects are the same.

- With lists, += generally extends the existing object, whereas a = a + values creates a new list and rebinds a.

Trap to remember: A shallow copy protects the outer container, not arbitrary nested objects.

Try it: Create aliases x and y for [1], execute x += [2], then x = x + [3]. Predict y after each line.

03 / Matrices and nested-container traps

- A matrix can be represented as a list of row lists. matrix[r][c] first selects a row, then a column.

- [[0] * columns] * rows repeats references to the same row.

- Tuple immutability prevents replacing its element references, but a tuple may contain a mutable list whose contents can change.

- For a rectangular matrix, transpose swaps row and column positions. zip(*matrix) groups corresponding row elements and returns tuples; by default it stops at the shortest row.

Trap to remember: [[0]] * n repeats references, not independent lists. Multiplication is safe for the row of immutable integers, but not for the outer repeated row.

Try it: Create a 3 x 3 identity matrix with independent rows.

Dictionaries & sets

Hashing, counting, grouping and fast membership

01 / Dictionaries and hashable keys

- A dict maps unique keys to values.

- Keys must be hashable: their hash must remain stable and equality must agree with hashing.

- d[k] raises KeyError if absent. d.get(k, default) returns a fallback without inserting. setdefault inserts a fallback when absent. in d checks keys; d.values() and d.items() expose values and key-value pairs.

- A hash table first uses a hash to narrow down where a key might be, then checks equality.

Trap to remember: get does not insert. Also, adding or removing keys while iterating a dictionary can raise RuntimeError; iterate a copy or collect changes first.

Try it: Build a histogram for [2,2,3,1,3,3] without Counter.

02 / Sets and choosing the right container

- A set stores distinct hashable elements and supports membership, union |, intersection &, difference -, and symmetric difference ^.

- {} creates an empty dictionary; set() creates an empty set. add inserts one element; update inserts elements from an iterable. remove raises KeyError when missing; discard does not.

- Use a list for ordered sequences with duplicates, tuple for a fixed record, dict for key-to-value association, and set for uniqueness and membership.

- A set trades memory for fast expected membership.

Trap to remember: Do not answer a question with a fixed set print order unless the question explicitly imposes one.

Try it: Find IDs present in yesterday=[1,2,3] but absent today=[2,4].

03 / Counting and grouping patterns

- Counting stores one aggregate per key.

- The two-sum pattern scans values while storing what has been seen.

- To group anagrams, choose a canonical key: a sorted character tuple, or a fixed-alphabet frequency tuple.

- When using a mutable default value, create a new object for each group. dict.fromkeys(keys, []) gives every key the same list, much like repeated matrix rows.

Trap to remember: Do not initialise all grouping buckets with the same shared list.

Try it: Group ["ant","apple","bat","bee"] by first letter.

Functions, scope & recursion

Contracts, call frames, shared state and recursive reasoning

01 / Parameters, return and contracts

- A function packages a reusable computation.

- A function without an executed value-return returns None.

- Parameters are local names bound to the argument objects.

- A pure function gives the same output for the same inputs without changing external state.

Trap to remember: A function that prints a result is not automatically usable as a numeric expression.

Try it: Write safe_mean(values), returning None for an empty sequence.

02 / Arguments and the mutable-default trap

- Arguments may be positional or named.

- *args collects additional positional arguments into a tuple; **kwargs collects additional keyword arguments into a dict.

- A default is a reference to the object at definition time.

- Prefer an explicit, small signature over accepting arbitrary arguments without validation.

Trap to remember: The bad example is intentionally wrong for independent collections; shared defaults can be deliberate, but should not be accidental.

Try it: Implement total(*values, offset=0) and call it with an unpacked list.

03 / Scope, closures and name lookup

- For ordinary function bodies, name lookup follows Local, Enclosing function scopes, Global module scope, and Built-ins (LEGB).

- Reading a local name before it has been bound raises UnboundLocalError. global targets a module-level name; nonlocal targets an already existing binding in an enclosing function.

- A closure retains access to its enclosing bindings after the outer function returns.

- Capture a value deliberately with a default parameter, or build each closure in a helper call with a distinct local binding.

Trap to remember: A name is not automatically global because it has not yet been assigned on the current execution path. Assignment anywhere in the function normally classifies it as local.

Try it: Create make_multiplier(k) returning a function that multiplies its input by k.

04 / Recursion: descent, base case, unwind

- A recursive function solves a problem using smaller instances of itself.

- Every active call has its own local parameters and pending work.

- Distinguish printing before a recursive call from printing after it.

- A linear-depth recursive routine uses O(n) call-stack space even if each frame has only a few variables.

Trap to remember: Do not forget the recursive call consumes stack space. Raising the recursion limit is not a substitute for choosing an appropriate algorithm.

Try it: Write recursive sum_digits(n) for nonnegative integers, using // and %.

05 / Repeated work and memoisation

- A branching recursion can solve the same subproblem many times.

- Memoisation stores answers by input so each distinct subproblem is computed once. functools.cache provides an unbounded memo table for hashable arguments.

- Bottom-up dynamic programming computes smaller results first.

- Euclid's gcd recursion uses gcd(a,b)=gcd(b,a%b), ending at b=0.

Trap to remember: A cache is useful only when repeated calls reuse results; it also consumes memory and requires hashable inputs.

Try it: Compare fib_loop with fib for every n from 0 through 20.

Write expressive Python

Comprehensions, iterators, generators and sorting keys

01 / Comprehensions and iteration helpers

- A comprehension describes a new collection from an iterable: [expression for x in source if condition].

- Nested comprehension clauses follow the same left-to-right order as nested loops. [x for row in matrix for x in row] flattens rows.

- enumerate supplies index-value pairs; zip aligns multiple iterables and by default stops at the shortest. zip(..., strict=True) (Python 3.10+) raises if lengths differ.

- any short-circuits at the first true item; all at the first false item. any([]) is False and all([]) is True. sum can count Boolean predicates because True and False act as 1 and 0 in arithmetic.

Trap to remember: Dense comprehensions can hide logic. Use an ordinary loop when there are several updates, exceptions, or side effects.

Try it: Flatten [[1,2],[],[3,4]] and keep only even values.

02 / Iterators and generators

- An iterable can provide an iterator with iter().

- A generator function contains yield.

- A generator expression uses parentheses and computes values lazily.

- map, filter, zip and enumerate produce lazy iterator objects in Python 3.

Trap to remember: A generator is not an automatically reusable list. Saving it does not save all its produced values.

Try it: Write a generator yielding running totals of [2,5,-1].

03 / Functions as values and stable sorting

- Functions can be passed as arguments, returned, or stored in containers. lambda creates a function from one expression; def is clearer for multi-step logic.

- sorted returns a new list; list.sort mutates.

- To sort by descending score then ascending name, use key=lambda row: (-row[1], row[0]). reverse=True reverses the full ordering, which is different when only one criterion should descend.

- min and max also accept key and, for an iterable, a default for empty input.

Trap to remember: reverse=True does not mean reverse only the first component of a tuple key.

Try it: Sort ["pear","fig","apple","kiwi"] by length, breaking ties alphabetically.

Build reliable programs

Exceptions, tests, modules and data files

01 / Exceptions and debugging deliberately

- Read a traceback from its final exception line, then inspect the relevant frames and source lines.

- try handles only exceptions raised in its suite. except should name the specific exceptions you can meaningfully handle. else runs when the try suite succeeds; finally runs during normal exit and exception unwinding.

- raise communicates a failed contract.

- Debug systematically: reproduce with the smallest input, state the expected result, inspect intermediate values using repr, change one hypothesis at a time, and add a regression test.

Trap to remember: Catching Exception around an entire application can conceal coding errors. Handle failures close to where recovery makes sense.

Try it: Convert a list of strings into integers, collecting invalid entries separately.

02 / Tests and edge cases

- A test compares observed behaviour to a specification.

- Use assertions for small development checks and unittest or another test runner for a growing project.

- Properties can catch errors beyond fixed examples: sorting should preserve length and element counts, reversing twice should recover the sequence, and normalising an already normalised value should leave it unchanged.

- Use a simple trusted method as an oracle for small inputs, then compare a faster implementation.

Trap to remember: Do not generate expected answers using the same buggy algorithm under test.

Try it: Write tests for a palindrome function, including empty text and mixed case.

03 / Files, JSON, CSV and context managers

- Use with open(path, mode, encoding="utf-8") as f to close a file even when processing raises an exception. r reads, w truncates or creates, a appends, and x creates only if absent.

- read loads content; iterating over a file reads lines progressively. write returns the number of characters written and does not append a newline automatically.

- json encodes structured objects such as lists and dicts.

- The browser lab uses a temporary virtual filesystem, separate from your computer.

Trap to remember: Opening an existing file with w erases its previous content. Know the mode before writing.

Try it: Round-trip a list of two student records through JSON and verify equality.

04 / Modules and a practical workflow

- A module is a Python file with its own namespace. import math binds the module name; from math import sqrt binds a particular attribute.

- A script commonly puts orchestration in main() and calls it under if __name__ == "__main__".

- Use python -m venv .venv to create a project environment and python -m pip install package to install into the chosen interpreter.

- Develop in a cycle: write a small specification, implement one function, run tests, inspect failures, refactor, and document.

Trap to remember: A module name can be shadowed by a local file or variable of the same name.

Try it: Extract a reusable clamp(x,low,high) function with validation and three tests.

Objects & advanced tools

Classes, data modelling, decorators and resource lifetimes

01 / Classes and independent state

- A class defines a type whose instances combine state and behaviour. __init__ initialises a new instance; self refers to the instance receiving a method call.

- Instance attributes belong to one object; class attributes are shared through the class.

- Methods can enforce invariants: a bank balance or queue length should stay valid after each operation.

- Use a class when related state and operations form a coherent concept.

Trap to remember: Defining minutes=[] on the class would share one list unless each instance explicitly overrides it.

Try it: Add an average method that returns zero when no sessions exist.

02 / Protocols, composition and inheritance

- Python often cares about supported behaviour rather than an exact class.

- Special methods let objects participate in language operations: __len__, __iter__, __repr__, __eq__, and others.

- Composition places one object inside another to reuse behaviour.

- Equality and identity remain different for objects.

Trap to remember: A frozen dataclass containing a list still contains a mutable list; shallow freezing does not recursively freeze everything.

Try it: Model a Point(x,y) record and compare two equal coordinate pairs.

03 / Decorators, context and advanced boundaries

- A decorator takes a function (or class) and returns a replacement. @decorate above def f is roughly f = decorate(f).

- A closure lets the wrapper remember the original function.

- A context manager brackets an operation with setup and cleanup. with is useful for files and resources; contextlib.contextmanager can implement one using a generator with a single yield and cleanup in finally.

- Advanced Python also includes async/await, concurrency, descriptors and metaclasses.

Trap to remember: A wrapper that forgets return silently converts successful function results into None.

Try it: Write a decorator that rejects negative first arguments and preserves the original result.

Reason about algorithms

Complexity, invariants, prefix sums and search

01 / Time and space from code

- Choose the input size first: n might be list length, number of vertices, or number of digits.

- Sequential loops add costs; nested loops do not always multiply by n blindly.

- Under the standard list model, indexing and len are O(1), append is amortised O(1), and front insertion/removal is O(n).

- Auxiliary space excludes the input unless stated otherwise.

Trap to remember: O(n^2) does not mean exactly n^2 operations. Also, O(n) can be a valid but loose upper bound for an O(1) routine.

Try it: Count iterations of repeated integer halving for n=1,2,8,9.

02 / Prefix sums and sliding windows

- For repeated range-sum queries, precompute prefix[0]=0 and prefix[i+1]=prefix[i]+a[i].

- A fixed-width sliding window updates the previous sum by removing the outgoing value and adding the incoming value.

- Validate k: it should be positive and no larger than the input length for the usual full-window problem.

- Two pointers can also solve variable-window problems, but the rule for moving pointers needs a proof.

Trap to remember: A half-open [left,right) query excludes right. Prefix arrays have n+1 entries for this convention.

Try it: Implement maximum sum of any two consecutive elements in [-8,-3,-5].

03 / Linear and binary search

- Linear search examines elements until it finds a match.

- Binary search needs an ordered search space and a predicate that divides it consistently.

- A lower-bound search returns the first index where a[i]>=target.

- Each iteration shrinks the interval, giving O(log n) comparisons.

Trap to remember: Binary search on unsorted data is not correct merely because the code terminates.

Try it: Test lower_bound against bisect_left on empty, singleton and duplicate-containing lists.

Data structures in Python

Stacks, queues, linked nodes, trees and graph traversal

01 / Stack and queue behaviour

- A stack is last-in, first-out.

- Stacks model pending work, undo history and matched delimiters.

- Queues model tasks waiting in arrival order and breadth-first exploration.

- The abstract data type specifies behaviour; a representation implements it.

Trap to remember: Ignoring the empty-stack case can turn an invalid string into an IndexError rather than a clean False.

Try it: Use a deque to simulate serving three named tasks in arrival order.

02 / Linked nodes and binary trees

- A singly linked list stores a value and a reference to the next node.

- A binary tree has at most two children per node.

- A binary search tree maintains an ordering rule: left keys are smaller and right keys larger, with a declared duplicate policy.

- A heap is different from a BST: the parent priority is no greater than its children in a min-heap, but sibling/subtree keys are not globally ordered. heapq supports O(log n) push/pop and O(n) heap construction.

Trap to remember: A BST is not automatically balanced. O(log n) search requires a logarithmic-height tree.

Try it: Rewrite inorder as a generator using yield from.

03 / Graphs, visited state and BFS

- A graph consists of vertices and edges.

- Breadth-first search uses a queue and discovers vertices in distance layers.

- Depth-first search explores deeply using recursion or an explicit stack.

- BFS does not solve arbitrary weighted shortest paths.

Trap to remember: A graph may be disconnected. A traversal from one start visits only its reachable component.

Try it: Add an isolated vertex E and verify it is absent from bfs(graph,"A").

Build, explain, revise

Three practical projects and a route to exam readiness

01 / Project: a study-session analyser

- Goal: take records with topic and minutes, reject malformed values, and report total minutes per topic.

- Define the contract: each row has a string topic and a nonnegative integer duration.

- The aggregation is a dict keyed by normalised topic.

- Extend the project with CSV input, JSON output and a command-line interface after the pure aggregation function works.

Trap to remember: A silent skip hides data quality problems. Keep a useful error trail.

Try it: Add tests for an empty list, an empty topic, a Boolean duration, and repeated normalised topics.

02 / Project: a text-frequency report

- Goal: extract words, count them, and display a deterministic top-k report.

- Use re.findall to extract tokens, casefold to normalise, Counter to count, and a tuple sort key for frequency descending then word ascending.

- Complexity has several stages: tokenisation is proportional to text length for this simple pattern, counting is expected linear in token count, and sorting u unique words costs O(u log u).

- Extensions: ignore a configurable stop-word set, read UTF-8 files, export JSON, and compare reports from two documents.

Trap to remember: A plain split() retains punctuation, so "python" and "python." may be counted separately.

Try it: Add an optional stop-word set and verify that uppercase stop words are handled under a documented policy.

03 / Project: your exam tracing workflow

- For an output question, first identify types and aliases.

- For a loop question, record the state immediately before the body, any output inside the body, and the update.

- Predict before running.

- Completion criteria: explain each lesson without notes; pass its check; implement its hands-on task from a blank editor; solve unseen mixed questions; and justify your answer under time pressure.

Trap to remember: Do not assume an output option is correct because it resembles a previously seen example. Trace this exact program.

Try it: Write your own short program containing one alias, one copy, and one rebinding. Predict every output and explain the arrows.

Before you call it mastered

- Trace assignment, mutation and copying with object arrows.
- Compute negative // and % using a = q*b + r.
- Resolve slicing with explicit indices and excluded stops.
- Explain mutable defaults, LEGB scope and closure capture.
- Draw recursive frames and account for stack space.
- Distinguish iterables from consumed iterators.
- Test empty, singleton, duplicate, negative and boundary inputs.
- Choose a container and justify its operation costs.
- Predict an unseen output before running code.
- Reattempt missed questions after a delay, without reading the solution.

Primary sources and verification

Official syllabus: https://gate2027.iitm.ac.in/exam_papers_and_syllabus

DA syllabus PDF: https://gate2027.iitm.ac.in/static/doc/GATE2027_Syllabus/DA_GATE2027_Syllabus.pdf

Python language reference: https://docs.python.org/3/reference/

Python tutorial: https://docs.python.org/3/tutorial/

PYQ paper and answer-key links appear beside each question in the interactive guide. Consult the exact original question before answering; paraphrased topic descriptions are only navigation aids.

Validation: all 41 worked examples and all 41 hands-on solutions were executed with Python; expected example outputs were compared automatically. Conceptual explanations and scope still require thoughtful use alongside official exam material.